

Vector Space Models & Word Embeddings

Naeemullah Khan

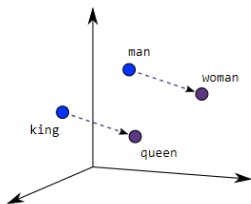
naeemullah.khan@kaust.edu.sa



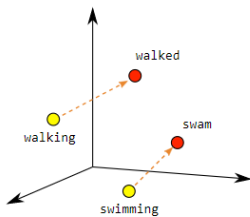
جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

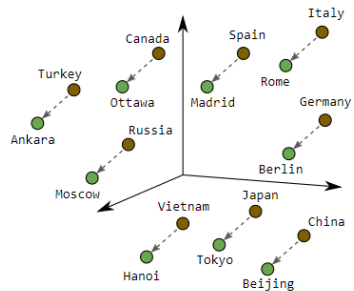
August 9, 2026



Male-Female



Verb Tense



Country-Capital

Source: adapted from the TensorFlow word2vec tutorial figure, <https://www.tensorflow.org/images/linear-relationships.png> (Country-Capital panel redrawn).

1. Motivation
2. Learning Outcomes
3. Introduction
4. Word-by-Word vs Word-by-Document Designs
5. Similarity Measures
 1. Euclidean Distance
 2. Cosine Similarity
 3. Manipulating word vectors
6. Word Embedding: One-Hot vs. Dense Vectors
 1. One-Hot Encoding
 2. Dense Embeddings
7. Embedding Method: Word2Vec

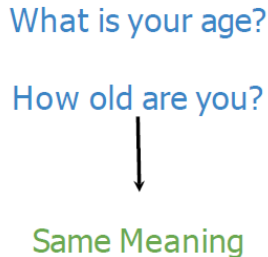
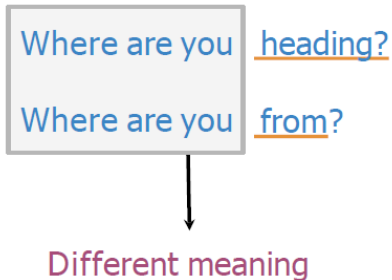
1. Word2Vec – Continuous Bag-of-Words (CBOW)
2. Word2Vec – Skip-Gram
- 3.
- 4.
- 5.
- 6.
- 7.
8. GloVe – Global Vectors
9. fastText – Subword Embeddings
10. Limitations of Vector Space Models
11. Summary
12. References

- ▶ Text is symbolic; machines require numerical representations to process it.
- ▶ Classical NLP struggled with sparse, high-dimensional vectors.
- ▶ Vector Space Models (VSMs) and Word Embeddings represent words in a continuous vector space.
- ▶ These models capture semantic meaning and relationships geometrically.
- ▶ Basis for modern NLP methods including Transformers.

- ▶ Understand and implement basic Vector Space Models.
- ▶ Differentiate between word-by-word and word-by-document designs.
- ▶ Use similarity measures to compute semantic relatedness.
- ▶ Explain the rationale behind One-Hot vs. Dense word vectors.
- ▶ Understand and implement basic Word2Vec models (CBOW and Skip-gram).
- ▶ Appreciate the improvements of GloVe and fastText over earlier models.

Vector Space Model: **Introduction**

Why learn vector space models?



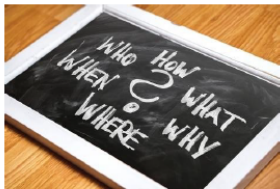
Source: DeepLearning.AI, *NLP with Classification and Vector Spaces* (Coursera), Week 3: Vector Space Models.

- ▶ Words and sentences can have different meanings depending on context.
- ▶ Vector space models help capture semantic similarity and differences.
- ▶ Useful for tasks like paraphrase detection, question answering, and information retrieval.

- ▶ VSM represents words or documents as vectors in an n -dimensional space.
- ▶ Based on the **distributional hypothesis**: Words that occur in similar contexts have similar meanings.
- ▶ Forms the foundation for information retrieval, document classification, and word embeddings.

- ▶ A **term-document matrix**: Rows represent words (terms), columns represent documents (or vice versa).
- ▶ Each cell contains a value such as:
 - Term Frequency (TF)
 - TF-IDF (Term Frequency-Inverse Document Frequency)
 - Co-occurrence count
- ▶ The matrix is typically high-dimensional and sparse:
 - Size = $|\text{Vocabulary}| \times |\text{Documents}|$

- ▶ You eat *cereal* from a *bowl* → Capturing semantic similarity (paraphrase understanding)
- ▶ You *buy* something and someone else *sells* it → Capturing relational meaning (analogies)



Information Extraction



Machine Translation



Chatbots

Source: DeepLearning.AI, *NLP with Classification and Vector Spaces* (Coursera), Week 3: Vector Space Models.

Word-by-Word vs Word-by-Document Designs

- ▶ Co-occurrence $\rightarrow$ Vector representation
- ▶ Relationships between words/documents

Number of times they *occur together within a certain distance* k

I like simple data
I prefer simple raw data

k=2

	simple	raw	like	I
data	2	1	1	0

n

Source: DeepLearning.AI, *NLP with Classification and Vector Spaces* (Coursera), Week 3: Vector Space Models.

Number of times a word *occurs within a certain category*

	Corpus		
	Entertainment	Economy	Machine Learning
data	500	6620	9320
film	7000	4000	1000

Source: DeepLearning.AI, *NLP with Classification and Vector Spaces* (Coursera), Week 3: Vector Space Models.

► Word-by-Word:

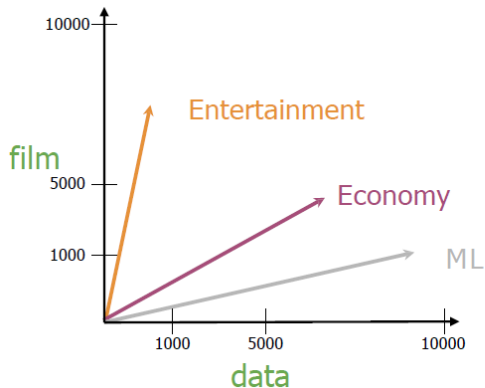
- Matrix built from co-occurrence counts between words.
- Captures semantic similarity directly.
- Better for word similarity tasks.

► Word-by-Document:

- Matrix built from word frequencies in documents.
- Useful for document classification and search.
- Better for document-level tasks.

Tradeoffs: Choice depends on the task: word similarity vs. document analysis.

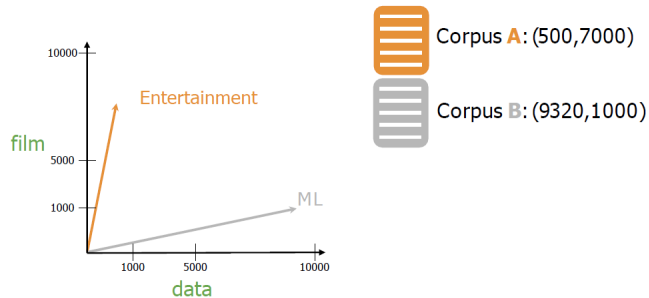
Similarity Measures



	Entertainment	Economy	ML
data	500	6620	9320
film	7000	4000	1000

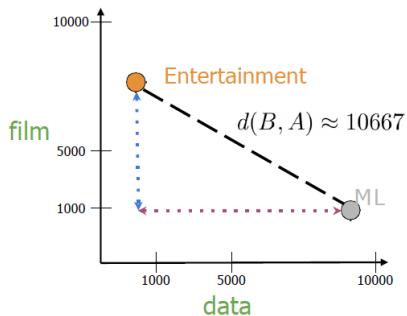
Measures of "similarity:"
Angle
Distance

Source: DeepLearning.AI, *NLP with Classification and Vector Spaces* (Coursera), Week 3: Vector Space Models.



Source: DeepLearning.AI, *NLP with Classification and Vector Spaces* (Coursera), Week 3: Vector Space Models.

- Measures the straight-line distance between two points in space.
- Formula: $d(\mathbf{x}, \mathbf{y}) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$
- Sensitive to the scale (magnitude) of the vectors.



Corpus **A**: (500,7000)



Corpus **B**: (9320,1000)

$$d(B, A) = \sqrt{(B_1 - A_1)^2 + (B_2 - A_2)^2}$$
$$c^2 = a^2 + b^2$$

$$d(B, A) = \sqrt{(-8820)^2 + (6000)^2}$$

Source: DeepLearning.AI, *NLP with Classification and Vector Spaces* (Coursera), Week 3: Vector Space Models.

Euclidean Distance for n-dimensional vectors

	data	$\vec{w}$ boba	$\vec{v}$ ice-cream
AI	6	0	1
drinks	0	4	6
food	0	6	8

$$\begin{aligned} &= \sqrt{(1 - 0)^2 + (6 - 4)^2 + (8 - 6)^2} \\ &= \sqrt{1 + 4 + 4} = \sqrt{9} = 3 \end{aligned}$$
$$d(\vec{v}, \vec{w}) = \sqrt{\sum_{i=1}^n (v_i - w_i)^2} \longrightarrow \text{Euclidean Norm of } (\vec{v} - \vec{w})$$

Source: DeepLearning.AI, *NLP with Classification and Vector Spaces* (Coursera), Week 3: Vector Space Models.

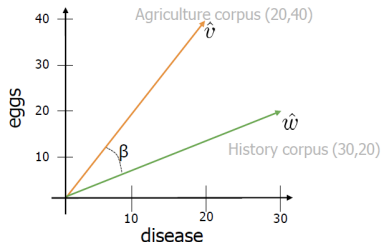
```
# Create numpy vectors v and w
v = np.array([1, 6, 8])
w = np.array([0, 4, 6])

# Calculate the Euclidean distance d
d = np.linalg.norm(v-w)

# Print the result
print("The Euclidean distance between v and w is: ", d)
```

The Euclidean distance between v and w is: 3

Source: DeepLearning.AI, *NLP with Classification and Vector Spaces* (Coursera), Week 3: Vector Space Models.



$$\hat{v} \cdot \hat{w} = \|\hat{v}\| \|\hat{w}\| \cos(\beta)$$

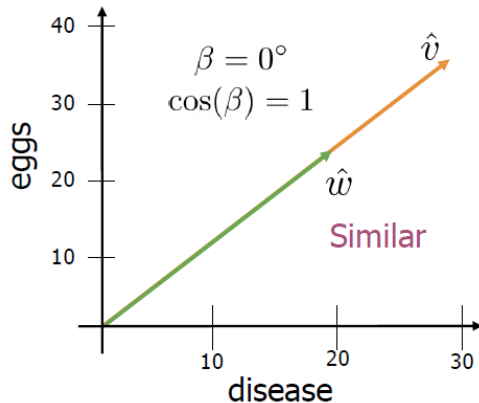
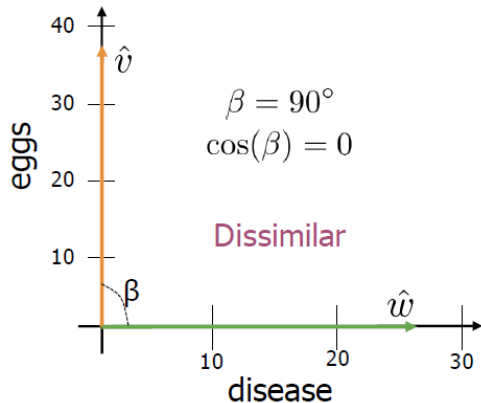
$$\cos(\beta) = \frac{\hat{v} \cdot \hat{w}}{\|\hat{v}\| \|\hat{w}\|}$$

$$= \frac{(20 \times 30) + (40 \times 20)}{\sqrt{20^2 + 40^2} \times \sqrt{30^2 + 20^2}} = 0.87$$

Source: DeepLearning.AI, *NLP with Classification and Vector Spaces* (Coursera), Week 3: Vector Space Models.

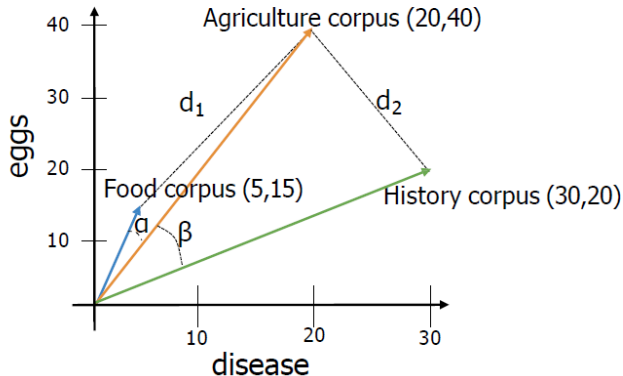
- Measures the cosine of the angle between two vectors.
- Formula: $\text{cosine}(\mathbf{x}, \mathbf{y}) = \frac{\mathbf{x} \cdot \mathbf{y}}{\|\mathbf{x}\| \|\mathbf{y}\|}$
- Ranges from -1 (opposite) to 1 (same direction).
- Less sensitive to magnitude, focuses on orientation.

Cosine Similarity (cont.)



Source: DeepLearning.AI, *NLP with Classification and Vector Spaces* (Coursera), Week 3: Vector Space Models.

- Cosine similarity when corpora are different sizes



Euclidean distance: $d_2 < d_1$

Angles comparison: $\beta > \alpha$

The cosine of the angle
between the vectors

Source: DeepLearning.AI, *NLP with Classification and Vector Spaces* (Coursera), Week 3: Vector Space Models.



USA



Washington
DC



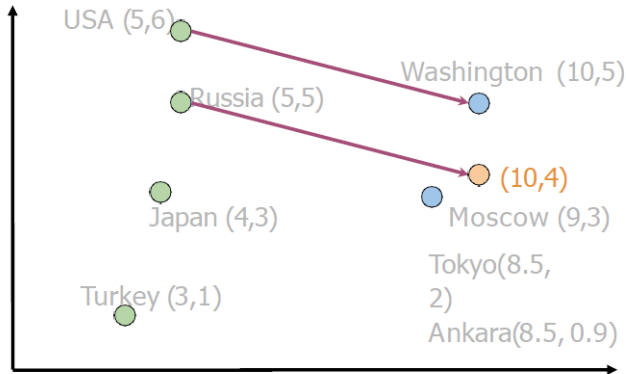
Russia



?

Source: DeepLearning.AI, *NLP with Classification and Vector Spaces* (Coursera), Week 3: Vector Space Models.

Manipulating word vectors (cont.)



$$\text{Washington} - \text{USA} = \begin{bmatrix} 5 & -1 \end{bmatrix}$$

$$\text{Russia} + \begin{bmatrix} 5 & -1 \end{bmatrix} = \begin{bmatrix} 10 & 4 \end{bmatrix}$$



Moscow

[Mikolov et al, 2013, Distributed Representations of Words and Phrases and their Compositionality]

Slide source: DeepLearning.AI, *NLP with Classification and Vector Spaces* (Coursera), Week 3; the credit inside the figure is to Mikolov et al., 2013, for the analogy result it illustrates.

	$d > 2$		
oil	0.20	...	0.10
gas	2.10	...	3.40
city	9.30	...	52.1
town	6.20	...	34.3

How can you visualize if your representation captures these relationships?



oil & gas



town & city

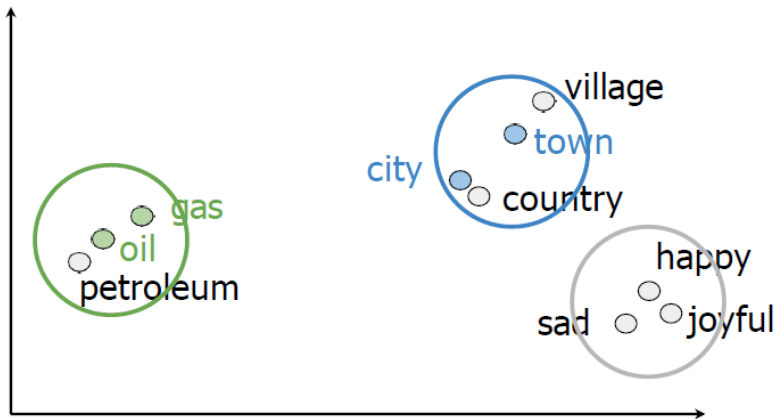
Source: DeepLearning.AI, *NLP with Classification and Vector Spaces* (Coursera), Week 3: Vector Space Models.

Visualization of word vectors (cont.)

	$d > 2$				$d = 2$	
oil	0.20	...	0.10	PCA →	oil	2.30 21.2
gas	2.10	...	3.40		gas	1.56 19.3
city	9.30	...	52.1		city	13.4 34.1
town	6.20	...	34.3		town	15.6 29.8

Source: DeepLearning.AI, *NLP with Classification and Vector Spaces* (Coursera), Week 3: Vector Space Models.

Visualization of word vectors (cont.)



Source: DeepLearning.AI, *NLP with Classification and Vector Spaces* (Coursera), Week 3: Vector Space Models.

Word Embedding: **One-Hot vs. Dense Vectors**

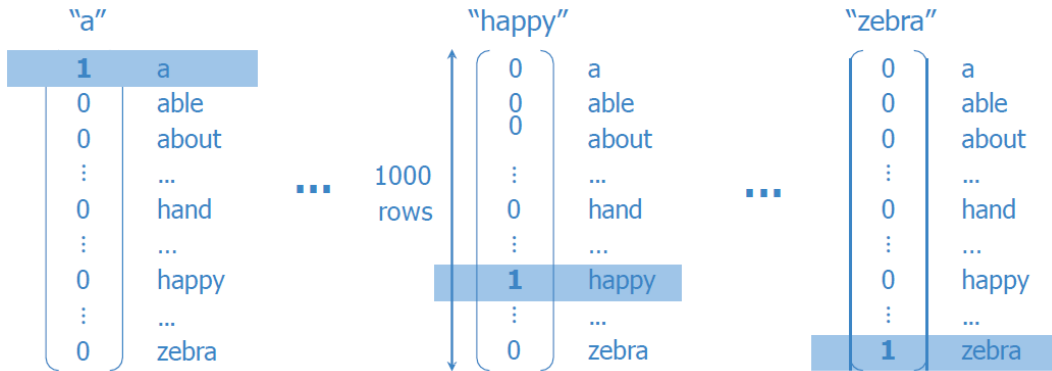
Word	Number
a	1
able	2
about	3
...	...
hand	615
...	...
happy	621
...	...
zebra	1000

Source: DeepLearning.AI, *NLP with Probabilistic Models* (Coursera), Week 4: Word Embeddings with Neural Networks.

- + Simple
- Ordering: little semantic sense

hand 615 < happy 621 < zebra 1000
?! ?!

Source: DeepLearning.AI, *NLP with Probabilistic Models* (Coursera), Week 4: Word Embeddings with Neural Networks.



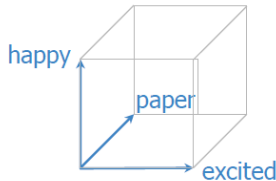
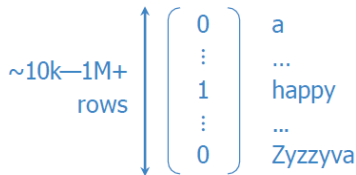
Source: DeepLearning.AI, *NLP with Probabilistic Models* (Coursera), Week 4: Word Embeddings with Neural Networks.

One-hot vectors (cont.)

Word	Number			
a	1		$\begin{pmatrix} 0 \\ 0 \\ 0 \\ \vdots \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{pmatrix}$	"happy"
able	2			
about	3			
...	...			
hand	615			
...	...			
happy	621			
...	...	1000	$\begin{pmatrix} \vdots \\ 0 \end{pmatrix}$	
zebra	1000			

Source: DeepLearning.AI, *NLP with Probabilistic Models* (Coursera), Week 4: Word Embeddings with Neural Networks.

- + Simple
- + No implied ordering
- Huge vectors
- No embedded meaning



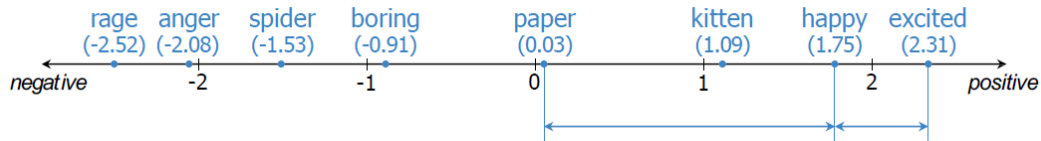
$$\begin{aligned} d(\text{paper}, \text{excited}) \\ &= d(\text{paper}, \text{happy}) \\ &= d(\text{excited}, \text{happy}) \end{aligned}$$

Source: DeepLearning.AI, *NLP with Probabilistic Models* (Coursera), Week 4: Word Embeddings with Neural Networks.

- ▶ Each word is represented as a unique vector.
- ▶ Vector has 1 at one position, 0 elsewhere.
- ▶ **Problems:**
 - High-dimensional and sparse.
 - No semantic meaning or similarity captured.

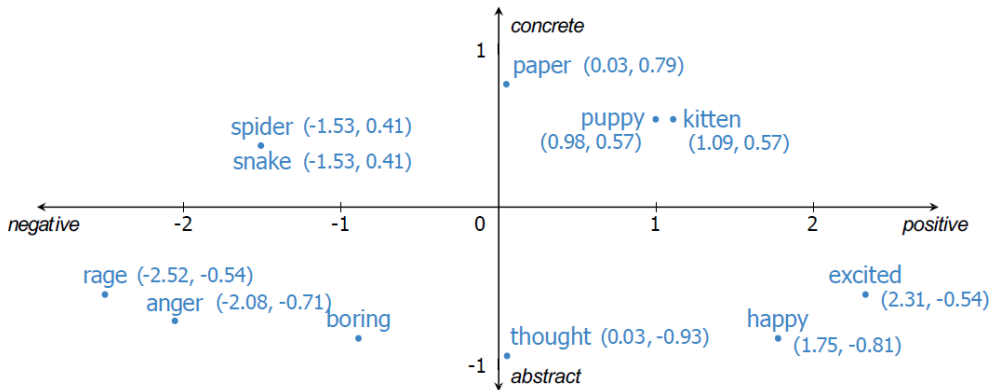
- ▶ Vectors are learned from data.
- ▶ Low-dimensional (e.g., 100–300).
- ▶ Capture semantic relationships.
- ▶ Example: "king" – "man" + "woman" $\approx$ "queen"

Meaning as vectors



Source: DeepLearning.AI, *NLP with Probabilistic Models* (Coursera), Week 4: Word Embeddings with Neural Networks.

Meaning as vectors (cont.)



Source: DeepLearning.AI, *NLP with Probabilistic Models* (Coursera), Week 4: Word Embeddings with Neural Networks.

+ Low dimension

+ Embed meaning

- e.g. semantic distance

forest $\approx$ tree forest $\not\approx$ ticket

- e.g. analogies

Paris:France :: Rome:?

"happy"

$\sim 100 - \sim 1000$
rows

$$\begin{pmatrix} 0.123 \\ \vdots \\ -4.059 \\ \vdots \\ 1.891 \end{pmatrix}$$

Source: DeepLearning.AI, *NLP with Probabilistic Models* (Coursera), Week 4: Word Embeddings with Neural Networks.

integers

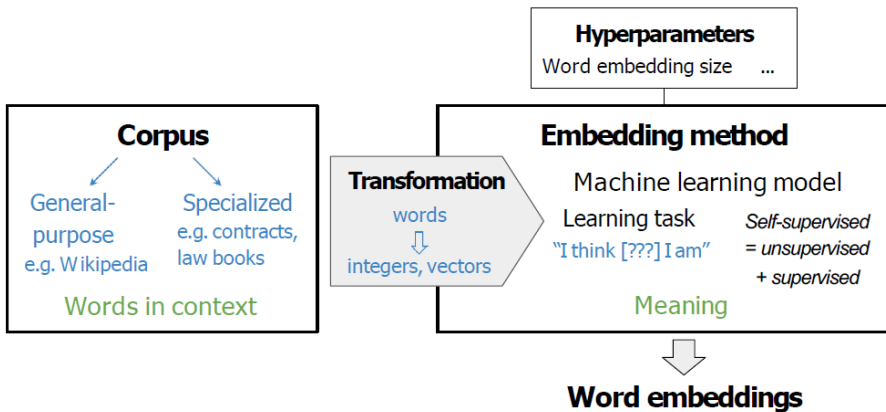
word vectors

one-hot vectors

word embedding vectors

"word vectors"
word embeddings

Source: DeepLearning.AI, *NLP with Probabilistic Models* (Coursera), Week 4: Word Embeddings with Neural Networks.



Source: DeepLearning.AI, *NLP with Probabilistic Models* (Coursera), Week 4: Word Embeddings with Neural Networks.

Embedding Method: **Word2Vec**

- ▶ Dense vector representations of words learned from context.
- ▶ Encode syntactic and semantic meaning.
- ▶ Vectors reflect relationships like synonymy, analogy, etc.

- ▶ **word2vec** (Google, 2013)
 - Continuous bag-of-words (CBOW)
 - Continuous skip-gram / Skip-gram with negative sampling (SGNS)
- ▶ **Global Vectors (GloVe)** (Stanford, 2014)
- ▶ **fastText** (Facebook, 2016)
 - Supports out-of-vocabulary (OOV) words

- ▶ Deep learning-based, contextual embeddings:
 - **BERT** (Google, 2018)
 - **ELMo** (Allen Institute for AI, 2018)
 - **GPT-2** (OpenAI, 2019)
- ▶ Tunable pre-trained models available

- ▶ Word2Vec learns word embeddings from large text corpora.
- ▶ Two main architectures:
 - Continuous Bag-of-Words (CBOW)
 - Skip-gram
- ▶ Both models use neural networks to predict words based on context.

- ▶ **Goal:** Predict target word from context.
- ▶ **Architecture:**
 - **Input:** Surrounding words (context)
 - **Output:** Target word
- ▶ Faster to train than Skip-gram.
- ▶ **Example:**
 - Context: “the ___ sat on the mat” → Predict “cat”

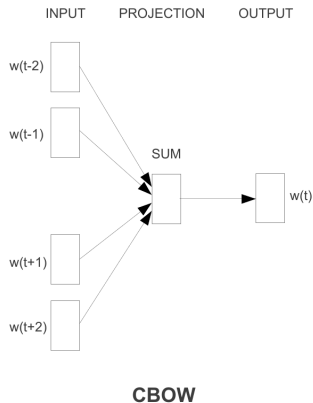


Figure 1: CBOW Architecture

Source: Mikolov et al., "Efficient Estimation of Word Representations in Vector Space", arXiv:1301.3781, Fig. 1 (left).

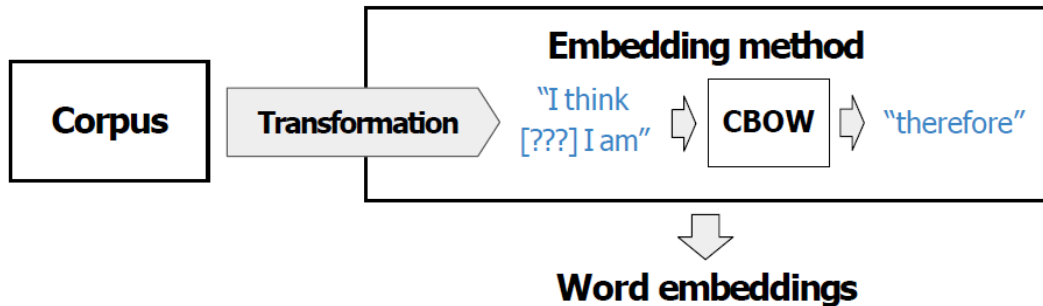
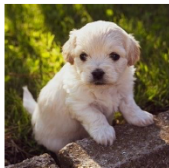


Figure 2: CBOW in the embedding pipeline: the model fills in the missing centre word.

Source: DeepLearning.AI, *NLP with Probabilistic Models* (Coursera), Week 4: Word Embeddings with Neural Networks.



The little _____ ? _____ is barking

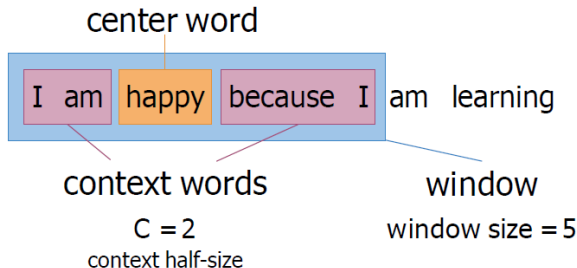


dog
puppy
hound
terrier

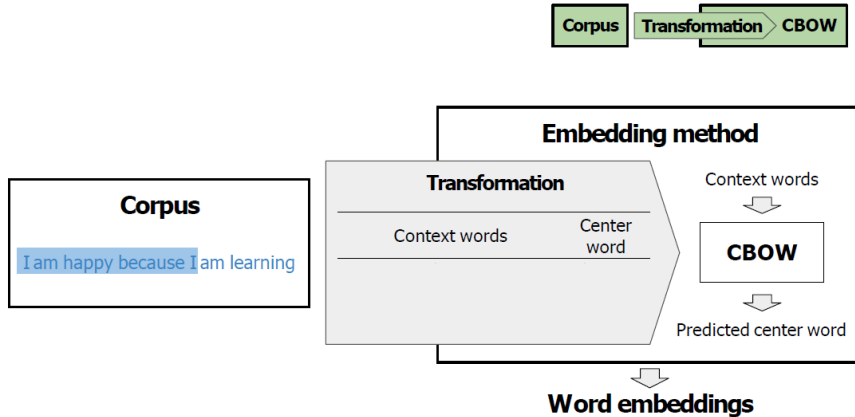
...

Source: DeepLearning.AI, *NLP with Probabilistic Models* (Coursera), Week 4: Word Embeddings with Neural Networks.

Creating a training example

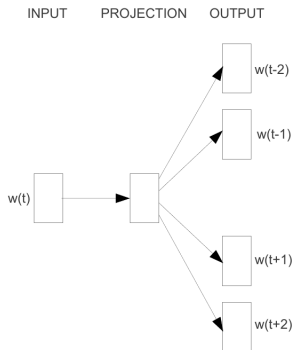


Source: DeepLearning.AI, *NLP with Probabilistic Models* (Coursera), Week 4: Word Embeddings with Neural Networks.



Source: DeepLearning.AI, *NLP with Probabilistic Models* (Coursera), Week 4: Word Embeddings with Neural Networks.

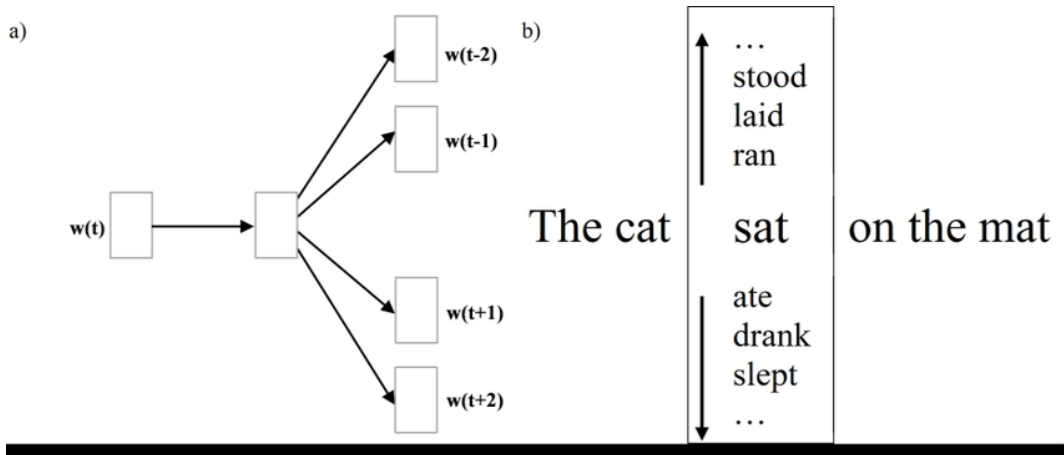
- ▶ **Goal:** Predict context words from a target word.
- ▶ **Architecture:**
 - **Input:** Target word
 - **Output:** Surrounding words (context)
- ▶ Slower to train, but better for rare words.
- ▶ **Example:**
 - Input: “cat” → Output: “the”, “sat”, “on”, “the”, “mat”



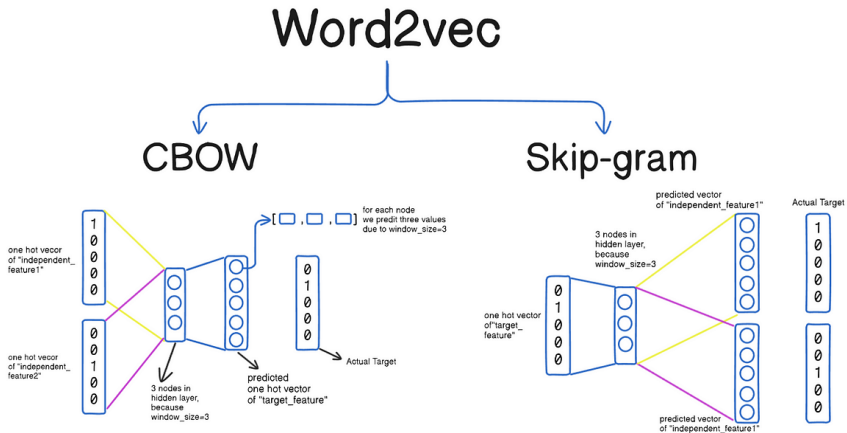
Skip-gram

Figure 3: Skip-Gram Architecture

Source: Mikolov et al., "Efficient Estimation of Word Representations in Vector Space", arXiv:1301.3781, Fig. 1 (right).



Source: K. Ezra, "Word Embedding [Complete Guide]", OpenGenus IQ (carries no credit there); panel (a) reproduces Mikolov et al., arXiv:1301.3781, Fig. 1 (right).



Source: F. Omarzai, "Word2Vec (CBOW, Skip-gram) In Depth", Medium, 1 August 2024.

- ▶ Does not consider sub-word information.
- ▶ Same vector for all senses of a word (polysemy issue).
- ▶ Ignores word order within the context window.

GloVe – Global Vectors

Proposed by: Pennington et al., 2014

- ▶ Combines global matrix factorization with local context windows.
- ▶ Captures co-occurrence statistics of words across entire corpus.
- ▶ Embeddings reflect ratios of co-occurrence probabilities.

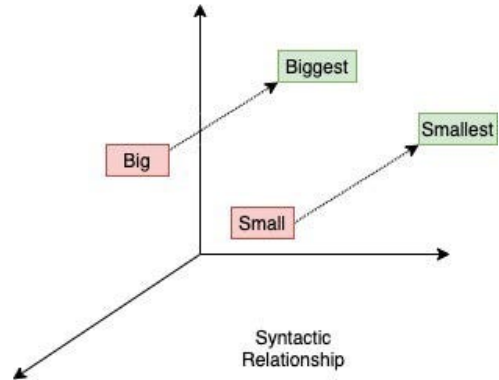
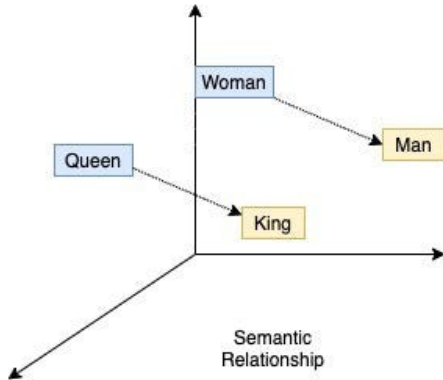
Loss function:

$$J = \sum_{i,j=1}^V f(X_{ij}) \left(w_i^\top \tilde{w}_j + b_i + \tilde{b}_j - \log X_{ij} \right)^2$$

where:

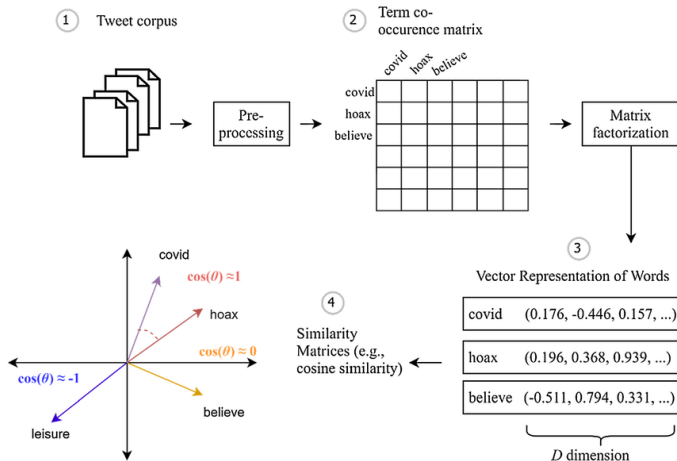
- ▶ X_{ij} : number of times word j occurs in the context of word i
- ▶ P_{ij} : probability of word j in the context of i
- ▶ $w_i, \tilde{w}_j$: word and context word vectors
- ▶ $b_i, \tilde{b}_j$: bias terms
- ▶ f : weighting function

GloVe – Global Vectors (cont.)



Source: N. Birajdar, "Word2Vec Research Paper Explained", Towards Data Science, 16 March 2021 (figure by the author).

GloVe – Global Vectors (cont.)



Source: Batzdorfer et al., "Conspiracy theories on Twitter: emerging motifs and temporal dynamics during the COVID-19 pandemic", *International Journal of Data Science and Analytics* 13(4), 2022, Fig. 1 (CC BY 4.0).

fastText – Subword Embeddings

- ▶ Developed by Facebook AI (2016)
- ▶ Builds on Word2Vec's skip-gram model by incorporating character n-grams
- ▶ Useful for morphologically rich languages and rare words
- ▶ Better handling of OOV (out-of-vocabulary) words: an unseen word still has n-grams, so its vector is the sum of those
- ▶ The same name also covers a companion supervised text classifier from the same group

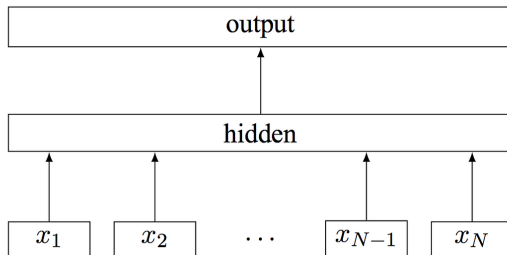
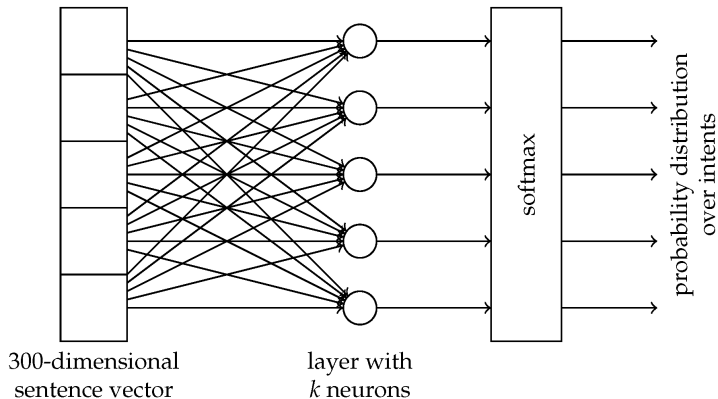


Figure 1: Model architecture of `fastText` for a sentence with N ngram features $x_1, \dots, x_N$. The features are embedded and averaged to form the hidden variable.

Source: Joulin et al., "Bag of Tricks for Efficient Text Classification", EACL 2017, Fig. 1. This is the `fastText` *classifier*, not the subword embedding model.



Source: Balodis & Dekšne, "FastText-Based Intent Detection for Inflected Languages", *Information* 10(5):161, 2019, Fig. 1.

- ▶ Word playing, first padded with boundary symbols: <playing>
- ▶ Character n-grams ($n=3$): <p1, pla, lay, ayi, yin, ing, ng>
- ▶ The whole word is kept as one extra sequence: <playing>
- ▶ Word vector = sum of n-gram vectors
- ▶ The boundary symbols separate affixes from the same letters elsewhere: <her> (the word *her*) is a different n-gram from *her* inside *where*
- ▶ In practice every n-gram with $3 \leq n \leq 6$ is used, not only $n = 3$

Feature	Word2Vec	GloVe	fastText
Trains on individual context windows	✓	—	✓
Trains on global co-occurrence counts	—	✓	—
Subword Info	—	—	✓
Handles OOV	—	—	✓

GloVe does use a context window (ten words each side, weighted $1/d$) to build the co-occurrence matrix. The distinction is what it trains on afterwards: the aggregated counts, not the individual windows.

► High Dimensionality:

- Vectors can become very high-dimensional, leading to computational inefficiency.
- Curse of dimensionality: distance metrics become less meaningful.

► Sparsity:

- One-hot vectors are sparse, leading to inefficiencies in storage and computation.
- Dense vectors mitigate this but still require large datasets for effective training.

► Lack of Context:

- Traditional VSMs do not capture word context effectively.
- Same word can have different meanings in different contexts (polysemy).

► Semantic Limitations:

- Cannot capture complex relationships like negation or antonymy.
- One surface form can carry unrelated senses (e.g., “bank” as a financial institution vs. “bank” of a river), yet it gets a single vector.

► Scalability:

- As vocabulary size increases, the term-document matrix becomes larger and more sparse.
- Requires significant computational resources for training and inference.

Summary

- ▶ **Contextual embeddings** (ELMo, BERT, GPT): Word vectors depend on sentence context.
- ▶ **Multilingual embeddings** and cross-lingual models.
- ▶ **Graph-based embeddings** (e.g., knowledge graph completion).
- ▶ **Hybrid embeddings**: Combining structured and unstructured data.

- ▶ Vector Space Models (VSMs) are foundational for modern NLP.
- ▶ Word embeddings capture semantic relationships in a continuous space.
- ▶ Word2Vec, GloVe, and fastText are key methods for generating word vectors.
- ▶ Dense embeddings outperform one-hot vectors in capturing meaning and relationships.
- ▶ Future work focuses on contextual, multilingual, and hybrid embeddings.

References

- [1] Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013).
Efficient Estimation of Word Representations in Vector Space.
arXiv:1301.3781.

- [2] Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013).
Distributed Representations of Words and Phrases and their Compositionality.
Advances in Neural Information Processing Systems 26 (NIPS 2013). arXiv:1310.4546.
(Introduces negative sampling, hence SGNS.)

- [3] Pennington, J., Socher, R., & Manning, C. D. (2014).
GloVe: Global Vectors for Word Representation.
EMNLP, 1532–1543.

- [4] Bojanowski, P., Grave, E., Joulin, A., & Mikolov, T. (2017).
Enriching Word Vectors with Subword Information.
TACL, 5, 135–146.

- [5] Joulin, A., Grave, E., Bojanowski, P., & Mikolov, T. (2017).
Bag of Tricks for Efficient Text Classification.
EACL, 427–431. arXiv:1607.01759.

- [6] Jurafsky, D., & Martin, J. H. (2026).
Speech and Language Processing (3rd ed. draft, release of January 6, 2026),
Chapter 5: Embeddings.
<https://web.stanford.edu/~jurafsky/slp3/>

- [7] Stanford NLP Slides.
<https://web.stanford.edu/class/cs224n>

- [8] DeepLearning.AI. Natural Language Processing Specialization (Coursera).
Course 1, *NLP with Classification and Vector Spaces*, Week 3: Vector Space Models; and
Course 2, *NLP with Probabilistic Models*, Week 4: Word Embeddings with Neural
Networks.
[https://www.deeplearning.ai/courses/
natural-language-processing-specialization/](https://www.deeplearning.ai/courses/natural-language-processing-specialization/)
(Lecture slides, Creative Commons; source of most figures in this deck.)

- [9] Facebook AI Research (FAIR) – fastText.
<https://fasttext.cc>

- [10] Goldberg, Y. (2016).
A Primer on Neural Network Models for NLP.
JAIR, 57, 345–420.

- [11] Chrupała, G., & Alishahi, A. (2019).
Correlating Neural and Symbolic Representations of Language.
Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, 2952–2962.

- [12] Balodis, K., & Dekšne, D. (2019).
FastText-Based Intent Detection for Inflected Languages.
Information, 10(5), 161.

- [13] Batzdorfer, V., Steinmetz, H., Biella, M., & Alizadeh, M. (2022).
Conspiracy Theories on Twitter: Emerging Motifs and Temporal Dynamics During the COVID-19 Pandemic.
International Journal of Data Science and Analytics, 13(4), 315–333.